

Outils de vérification formelle : Scyther

En un temps où la sécurité informatique est devenue d'une importance cruciale le logiciel scyther a vu le jour. Ce dernier représente un nouvel outil qui apporte de nombreux avantages dans le monde de la cryptographie et de la sécurité informatique en générale. C'est un logiciel gratuit très simple d'utilisation qui permet à lui seule de dégager les failles existantes dans des protocoles de communications en un temps record et sans aucun effort : en effet avant cet outil les spécialistes prenait beaucoup de temps avant de découvrir les imperfections des protocoles de sécurité mais trouvaient aussi beaucoup de difficulté.

1. Langage SPDL

Afin de détecter les failles d'un protocole donné, il faut d'abord le traduire en un langage « Security Protocol Description Language » SPDL. Voici la forme que doit avoir un protocole dans le SPDL :

```
// Partie déclaration  
  
protocol NOM_DU_PROTOCOLE  
  
(NOM_DES_AGENTS) {  
  
role AGENT1 { }  
  
role AGENT2 { }  
  
.....  
  
}  
  
//Partie concernant l'intrus
```

- Pour déclarer une fonction de hachage : `hashfunction hash;`
- Dans chaque rôle on va présenter les échanges entre l'initiateur et le répondeur, ces échanges seront décrits avec les primitives `send` « `send_1` » et `receive` « `recv_2` ». Par exemple : `send_1(A,S,{A,B,Na}Kas); recv_1(A,S,{A,B,Na}Kas);`
- Les données échangées peuvent être de différent type telle que :
 - Les nonces : `const n : Nonce ; var n : Nonce ;`
 - Les clés de session : `var Kab : SessionKey; const Kas:SessionKey;`
- Dans les systèmes asymétrique, les échanges entre les agents peuvent être chiffré avec la clé public ou la clé privé comme suit : `{na}pk(agent1)` ou `{na}sk(agent1)`.

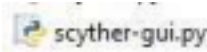
Outils de vérification formelle : Scyther

- Dans les systèmes symétriques, les échanges entre les agents doivent être chiffré avec une clé secret comme suit : $\{K_{ba}\}K_{bs}$; par exemple : `send_1(A,S,{A,B,Na}Kas)`;
- Dans chaque rôle on trouve les claims qui servent à exprimer les revendications des protocoles c'est-à-dire les points que doit offrir le protocole :
 - `claim_A1(A,Secret,Ks)`; Cela signifie que le claim 1 concerne l'agent A et que la clé Ks doit être secrète. Par exemple, `claim_B1(B,Secret,Kab)`;
 - `claim_A2(A,Nisync)` ; Cela signifie que les deux rôles doivent être exécuté de manière synchronisé c'est-à-dire exécuté dans l'ordre précisé. Par exemple : `claim_B3(B,Nisynch)`;
- **La partie intrus** : contient la déclaration de l'intrus, ce dernier est un agent extérieur non autorisé à accéder a certain paramètre de l'échange.
 - `const NOM_DE_L'INTRUS:Agent;`
 - `untrusted NOM_DE_L'INTRUS;`
 - `compromised SK(NOM_DE_L'INTRUS);`

Exemple :

```
const Kab:SessionKey;  
const eve :Agent ;  
untrusted eve ;  
compromised Kab(eve);
```

2. Exécution du protocole

Dans le dossier scyther, si on fait un double clic sur  la fenêtre suivante est affichée.



Outils de vérification formelle : Scyther

On commence par l'écriture du protocole avec langage SPDL par la suite on lance la vérification du protocole.

